

Consultas JPQL y transacciones

8. Consultas JPQL y transacciones

Añades una nueva vía de consulta mediante `@Query` y JPQL, pensada para consultas que resultan difíciles de expresar con un método derivado por nombre o con una Specification.

El problema: lo que ni el naming ni las Specifications resuelven bien

Imagina que quieres un ranking de editoriales, ordenado por cuántos libros tiene cada una en el catálogo. Prueba a resolverlo con lo que ya conoces:

Por qué ninguna de las dos llega

- Naming de método: no existe ninguna combinación de palabras que le pida a Spring Data "agrupa por editorial y ordena por el número de libros de cada grupo" — el naming deriva condiciones sobre campos, no agregaciones sobre una relación completa.
- Specifications: están pensadas para construir condiciones WHERE dinámicas y combinables — no para expresar un GROUP BY con ORDER BY sobre el resultado de una agregación. Se podría forzar con CriteriaBuilder, pero a cambio de una consulta rígida y bastante más difícil de leer que el propio JPQL.

Aquí entra @Query con JPQL

@Query con JPQL permite escribir consultas fijas —siempre con la misma estructura— que resultan demasiado elaboradas para expresarlas mediante el nombre de un método o combinando Specifications.

Qué es JPQL

JPQL (Jakarta Persistence Query Language) es el lenguaje de consultas de la propia especificación JPA. La idea de fondo es sencilla: en vez de preguntarle a la base de datos por sus tablas y columnas, le preguntas a JPA por tus clases Java y sus propiedades.

Lo más simple: una consulta sin relaciones de por medio

	Consulta
SQL	SELECT * FROM libro WHERE precio > 20
JPQL	SELECT l FROM Libro l WHERE l.precio > 20

Solo cambian dos cosas

Libro con mayúscula inicial (la clase Java, no la tabla libro), y l.precio en vez de solo precio (la propiedad del objeto l, no una columna suelta). El resto —SELECT, WHERE, el operador >— se escribe exactamente igual: JPQL toma prestada casi toda la sintaxis de SQL.

La diferencia se nota de verdad al cruzar una relación

	Consulta
SQL	<code>SELECT e.* FROM editorial e JOIN libro l ON l.editorial_id = e.id</code>
JPQL	<code>SELECT e FROM Editorial e JOIN e.libros l</code>

e.libros navega la relación, no repite la condición

En SQL escribes tú la condición de unión a mano (`ON l.editorial_id = e.id`), porque la base de datos no sabe nada de tus relaciones. En JPQL, `e.libros` navega la relación `@OneToMany` que ya has declarado en tu entidad `Editorial` — JPA ya conoce esa relación, así que no hace falta repetirla.

Las tres vías de consulta de Spring Data

Vía	Cuándo usarla
Naming de método (findByTitulo, findByEditorialId)	Consultas simples, fijas, sin lógica adicional.
Specifications	Filtros dinámicos, combinables en tiempo de ejecución.
@Query con JPQL	Consultas complejas y fijas — típicamente, agregaciones.

Qué es una agregación

Una agregación es una operación que resume varias filas en un único resultado: COUNT (cuántas), SUM (suma), AVG (media)... normalmente combinada con GROUP BY, que agrupa las filas antes de aplicar la agregación a cada grupo por separado.

Antes de llegar a la agregación, el uso más habitual de `@Query` es mucho más simple: una consulta con una condición que varía según quien la llama. Imagina que quieres los libros publicados por una editorial concreta, buscando por su nombre.

Una consulta con condición variable

```
public interface LibroRepository extends JpaRepository<Libro, Long> {  
  
    @Query("SELECT l FROM Libro l WHERE l.editorial.nombre = :nombreEditorial")  
    List<Libro> buscarPorNombreDeEditorial(@Param("nombreEditorial") String  
nombreEditorial);  
}
```

Dos pasos de navegación, y un parámetro que debe coincidir

`l.editorial.nombre` navega dos pasos con propiedades Java —primero a la relación editorial de Libro, luego a su propiedad nombre—, nunca con nombres de columna. `@Param("nombreEditorial")` conecta el `:nombreEditorial` de la consulta con el parámetro del método: el texto entre paréntesis tiene que coincidir exactamente con el que escribes tras los dos puntos en el JPQL.

Aquí, un error de escritura no espera a que ejecutes la consulta

⚠ Se valida al arrancar, no al ejecutar

Si escribieras `I.editorial.nombree` (una letra de más) o `:nombreEditoral` (un parámetro que no casa), Spring no esperaría a la primera petición para fallar: valida la sintaxis de cada `@Query JPQL` en cuanto arranca la aplicación, contra el propio modelo de tus entidades.

Es justo lo contrario de lo que le pasa a una `Specification`, que solo descubre un nombre de propiedad equivocado en el momento en que alguien la ejecuta de verdad, con una petición HTTP real. Un fallo en el arranque es más incómodo a corto plazo, pero mucho más seguro.

Un ejemplo completo: el ranking de editoriales

Imagina que quieres un ranking de editoriales ordenado por cuántos libros tiene cada una en el catálogo. Es una consulta fija, pero con JOIN, agregación y ordenación por el resultado de esa agregación — el caso típico de @Query.

La consulta con @Query

```
public interface EditorialRepository extends JpaRepository<Editorial, Long> {  
  
    @Query("""  
        SELECT e  
        FROM Editorial e  
        LEFT JOIN e.libros l  
        GROUP BY e  
        ORDER BY COUNT(l) DESC  
        """)  
    List<Editorial> rankingPorNumeroDeLibros();  
}
```

Una etiqueta descriptiva, no la consulta en sí

@Query le dice a Spring Data: "no derives la consulta a partir del nombre del método, usa literalmente este JPQL que te doy". Es la diferencia clave con el naming de método: ahí el nombre del método es la consulta; aquí el nombre es solo una etiqueta descriptiva, y la consulta real vive entera dentro de la anotación.

Desglosado, fragmento a fragmento

Fragmento	Qué hace
SELECT e	Devuelve el objeto Editorial completo — no una columna suelta, como harías en SQL.
FROM Editorial e	La entidad de partida, con el alias e para referirte a ella en el resto de la consulta.
LEFT JOIN e.libros l	Navega la relación Editorial → libros y mantiene también las editoriales sin ningún libro.
GROUP BY e	Agrupar, por cada editorial, todos los libros que comparten esa misma editorial.
ORDER BY COUNT(l) DESC	Ordena esos grupos por cuántos libros tiene cada uno, de mayor a menor.

Junta las cinco piezas

La consulta completa dice: «para cada editorial, cuenta sus libros y ordénalas de la que más tiene a la que menos».

Por qué LEFT JOIN, y no un JOIN normal

Se utiliza LEFT JOIN para que también aparezcan las editoriales que todavía no tienen libros. En ese caso, COUNT(I) vale 0 y la editorial queda al final del ranking.

```
Editorial con 5 libros → COUNT(I) = 5  
Editorial con 2 libros → COUNT(I) = 2  
Editorial sin libros   → COUNT(I) = 0
```

Cuando necesitas SQL directamente: `nativeQuery = true`

JPQL cubre la mayoría de casos, pero hay funcionalidad que directamente no existe en JPQL — las funciones de ventana (window functions) de SQL son el ejemplo más claro. La más simple es `ROW_NUMBER()`: numera cada fila, una a una, según el orden que le indiques.

ROW_NUMBER(), la función de ventana más simple

```
SELECT titulo, precio,  
       ROW_NUMBER() OVER (ORDER BY precio DESC) AS posicion  
FROM libro;
```

Esto no agrupa nada — sigues teniendo una fila por libro— pero cada fila lleva ahora un número de posición. `OVER (ORDER BY precio DESC)` es la "ventana": le dice a `ROW_NUMBER()` en qué orden mirar las filas para numerarlas.

Antes y después de ROW_NUMBER()

Libros de partida:

titulo	precio
El nombre del viento	24.99
Palabras de fuego	22.50
La sombra del viento	19.99
1984	12.99

Con la columna posicion añadida:

titulo	precio	pos.
El nombre del viento	24.99	1
Palabras de fuego	22.50	2
La sombra del viento	19.99	3
1984	12.99	4

Ahora aplica la misma idea al ranking de editoriales

Quieres que el propio ranking devuelva ya la posición numérica de cada editorial (1º, 2º, 3º...), en vez de calcularla tú en Java. Aquí la ventana no numera filas sueltas de libros, sino los grupos que ya forma el GROUP BY — así que combinas las dos cosas en la misma consulta. Eso ya es SQL de verdad, no JPQL.

El ranking con posición, en SQL nativo

```
@Query(value = """
    SELECT e.*, ROW_NUMBER() OVER (ORDER BY COUNT(l.id) DESC) AS posicion
    FROM editorial e LEFT JOIN libro l ON l.editorial_id = e.id
    GROUP BY e.id
    """, nativeQuery = true)
List<Object[]> rankingConPosicion();
```

El precio de pedirle a la BD algo que JPQL no puede expresar

`nativeQuery = true` le dice a Spring Data que ese texto no es JPQL — es SQL tal cual lo entiende tu gestor, con sus tablas, columnas y funciones específicas.

- Pierdes la independencia de motor que sí tenía JPQL — esta consulta solo funciona en gestores que soporten esa sintaxis.
- El resultado deja de ser una lista de `Editorial` — pasa a ser `List<Object[]>` (cada fila, tal cual la devuelve la base de datos), porque el motor no sabe mapear una columna calculada como `posicion` a ningún campo de tu entidad.

Transacciones al escribir con JPQL: @Modifying

Todo lo visto hasta ahora son lecturas, pero JPQL también puede ejecutar un UPDATE o un DELETE que afecte a varias filas sin cargarlas antes en Java una a una.

Ya has hecho algo parecido mediante un procedimiento almacenado. Ahora vas a conocer otra posibilidad: escribir la actualización directamente en el repository mediante @Query.

Aplicar un descuento a todos los libros de una editorial

```
@Modifying
@Transactional
@Query("UPDATE Libro l SET l.precio = l.precio * 0.9 WHERE l.editorial.id = :editorialId")
int aplicarDescuento(@Param("editorialId") Long editorialId);
```

@Modifying, y el int que devuelve

@Modifying le dice a Spring Data que esta consulta no es un SELECT, sino una operación que modifica datos. El método devuelve un int: el número de filas que se han actualizado. Por ejemplo, si el descuento se aplica a cinco libros, el método devolverá 5. Ninguna de esas filas necesita convertirse antes en un objeto Libro dentro de Java.

La diferencia con el procedimiento almacenado

Tanto este método como el procedimiento almacenado evitan recuperar todos los libros y recorrerlos con un bucle. La diferencia principal es dónde escribes la operación:

- En el procedimiento almacenado, la operación se guarda dentro de PostgreSQL.
- Con JPQL, la actualización se declara en el repository de la aplicación.

En este apartado utilizarás la segunda opción porque el objetivo es aprender a escribir consultas de modificación mediante `@Query` y `@Modifying`.

Aquí @Transactional no puede ser readOnly

⚠ Una transacción de escritura, sin readOnly

Un UPDATE o un DELETE modifica datos, por lo que necesita una transacción de escritura. Debes utilizar @Transactional a secas, igual que en los métodos create() y update() de tus servicios.

Ideas clave

- JPQL opera sobre entidades y propiedades Java, no sobre tablas y columnas SQL.
- Las tres vías de consulta de Spring Data: naming de método (simple y fijo), Specifications (dinámico), @Query/JPQL (complejo y fijo, típicamente agregaciones).
- @Param conecta un :parámetro del JPQL con un argumento del método — el nombre tiene que coincidir exactamente.
- Spring comprueba las consultas escritas con @Query al arrancar la aplicación; en una Specification, algunos errores solo aparecen al ejecutar la búsqueda.

Ideas clave (cont.)

- `@Query(nativeQuery = true)` escribe SQL literal cuando JPQL no llega — a cambio, pierdes independencia de motor y el resultado ya no son entidades, sino `Object[]` por fila.
- Una agregación (COUNT, SUM, AVG... con GROUP BY) resume varias filas en un resultado.
- `@Modifying` marca una `@Query` JPQL como escritura (UPDATE/DELETE); necesita `@Transactional` a secas, nunca `readOnly`, y devuelve el número de filas afectadas, no una lista.